

進階 RAG 技術

來源：<https://codelabs.developers.google.com/codelabs/production-ready-ai-with-gc/8-advanced-rag-methods/advanced-rag-methods?hl=zh-tw>

步驟數：11

「使用 Cloud SQL 和 Vertex AI 建構檢索增強生成 (RAG) 系統，並探索分塊策略、重新排序和查詢轉換。」

目錄

1. [簡介](#)
2. [專案設定](#)
3. [開啟 Cloud Shell 編輯器](#)
4. [啟用 API](#)
5. [建立虛擬環境並安裝依附元件](#)
6. [設定 PostgreSQL 適用的 Cloud SQL](#)
7. [第 1 部分：分塊策略](#)
8. [第 2 部分：重新排序](#)
9. [第 3 部分：查詢轉換](#)
10. [第 4 部分：端對端生成](#)
11. [結論](#)

1. 簡介

總覽

檢索增強生成 (RAG) 技術可根據外部知識，提升大型語言模型 (LLM) 的回覆品質。不過，如要建構可供正式環境使用的 RAG 系統，不只是簡單的向量搜尋即可。您必須最佳化資料的擷取方式、相關結果的排序方式，以及使用者查詢的處理方式。

在本實驗室中，您將使用 [PostgreSQL 適用的 Cloud SQL](#) (擴充 `pgvector`) 和 [Vertex AI](#)，建構強大的 RAG 應用程式。您將逐步瞭解三種進階技巧：

1. **分塊策略**：您會觀察到不同的文字分割方法 (字元、遞迴、符記) 如何影響擷取品質。
2. **重新排名**：您將實作 [Vertex AI Reranker](#)，以改善搜尋結果並解決「中間遺失」問題。
3. **查詢轉換**：您將使用 Gemini，透過 **HyDE** (假設文件嵌入) 和**逐步提示**等技術，最佳化使用者查詢。

學習內容

- 使用 `pgvector` 設定 PostgreSQL 適用的 Cloud SQL 執行個體。
- 建構資料擷取管道，使用多種策略將文字分塊，並將嵌入內容儲存在 Cloud SQL 中。
- 執行語意搜尋，並比較不同分塊方法產生的結果品質。
- 整合 Reranker，根據相關性重新排序檢索到的文件。
- 導入 LLM 輔助查詢轉換功能，提升模糊或複雜問題的檢索效果。

課程內容

- 如何搭配使用 **LangChain**、**Vertex AI** 和 **Cloud SQL**。
 - **字元、遞迴和詞元**文字分割器的影響。
 - 如何在 PostgreSQL 中實作**向量搜尋**。
 - 如何使用 **ContextualCompressionRetriever** 重新排序。
 - 如何實作 **HyDE** 和**逐步提示**。
-

步驟 2 · 約 0 分鐘

2. 專案設定

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

Google 帳戶

如果沒有個人 Google 帳戶，請[建立 Google 帳戶](#)。

請使用個人帳戶，而非公司或學校帳戶。

公司和學校帳戶可能設有限制，導致您無法啟用本實驗室所需的 API。

登入 Google Cloud 控制台

使用個人 Google 帳戶登入 [Google Cloud 控制台](#)。

啟用計費功能

兌換 Google Cloud 抵免額 (選用)

如要參加這個研討會，您需要有具備抵免額的帳單帳戶。使用本程式碼研究室頂端橫幅中的抵免額，即可開始。如果已連結帳單帳戶，可以略過這個步驟。

設定個人帳單帳戶

如果使用 Google Cloud 抵免額設定計費，可以略過這個步驟。

如要設定個人帳單帳戶，請[前往這裡在 Cloud 控制台中啟用帳單](#)。

注意事項：

- 完成本實驗室的 Cloud 資源費用應不到 \$1 美元。
- 您可以按照本實驗室結尾的步驟刪除資源，以免產生後續費用。
- 新使用者可獲得價值 **\$300 美元**的免費試用期。

建立專案 (選用)

如果沒有要用於本實驗室的現有專案，請[在這裡建立新專案](#)。

步驟 3 · 約 0 分鐘

3. 開啟 Cloud Shell 編輯器

1. 按一下這個連結，直接前往 [Cloud Shell 編輯器](#)
2. 如果系統在今天任何時間提示您授權，請點選「授權」繼續操作。

Authorize Cloud Shell

gcloud is requesting your credentials to make a GCP API call.

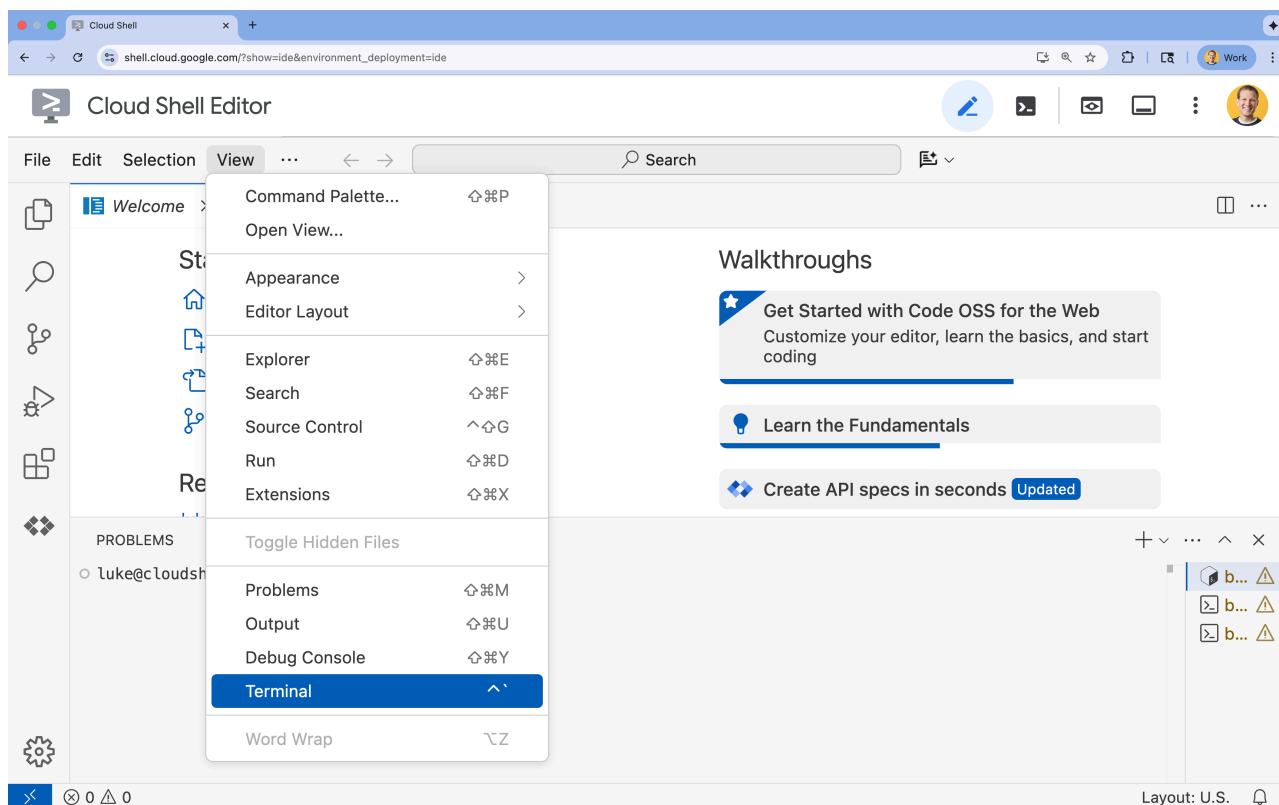
Click to authorize this and future calls that require your credentials.

REJECT

AUTHORIZE

3. 如果畫面底部未顯示終端機，請開啟終端機：

- 按一下「查看」
- 按一下「終端機」



4. 在終端機中，使用下列指令設定專案：

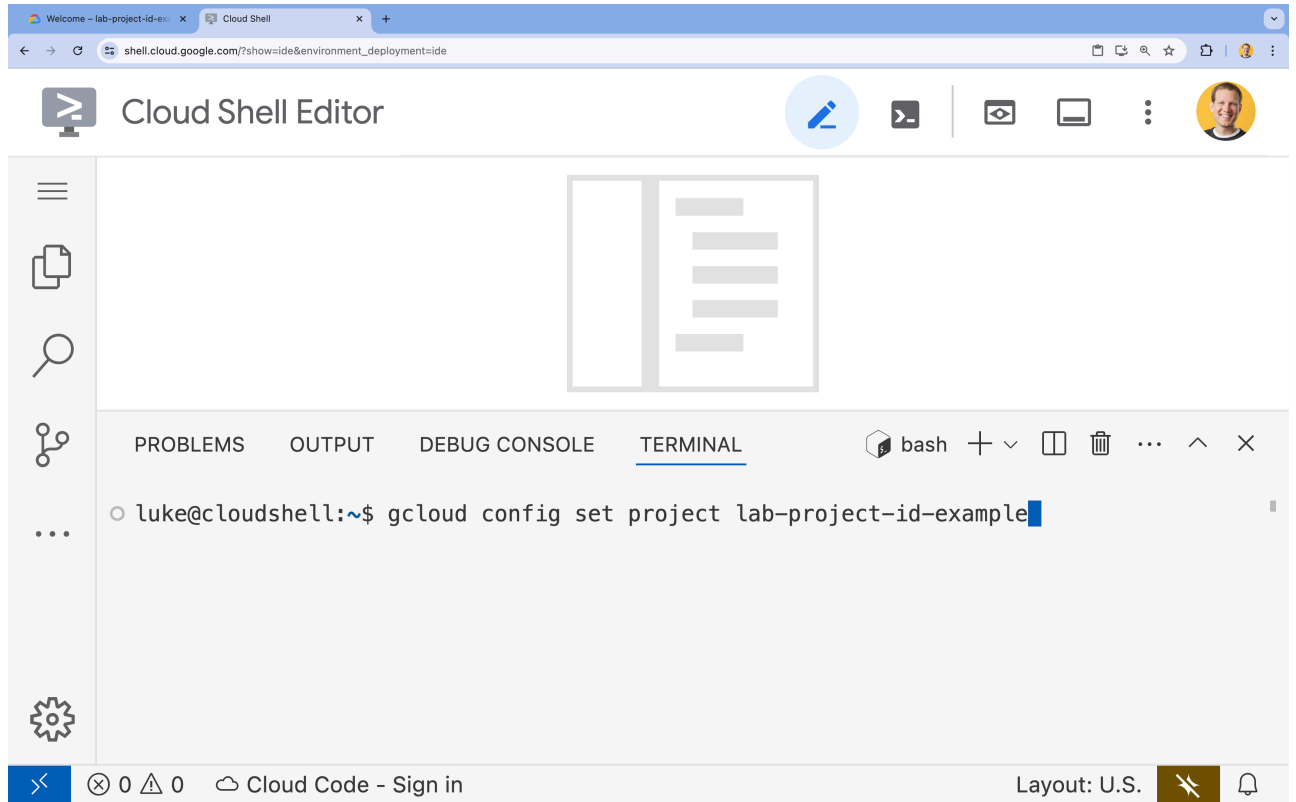
```
gcloud config set project [PROJECT_ID]
```

- 範例：

```
gcloud config set project lab-project-id-example
```

- 如果忘記專案 ID，可以使用下列指令列出所有專案 ID：

```
gcloud projects list
```



5. 您應會看到下列訊息：

```
Updated property [core/project].
```

如果看到 **WARNING** 並收到 **Do you want to continue (Y/n)?** 提示，表示您可能輸入了錯誤的專案 ID。按下 **n** 和 **Enter**，然後再次嘗試執行 **gcloud config set project** 指令。

4. 啟用 API

如要建構這項解決方案，您需要為 Vertex AI、Cloud SQL 和重新排序服務啟用多個 Google Cloud API。

1. 在終端機中啟用 API：

```
gcloud services enable \  
  aiplatform.googleapis.com \  
  sqladmin.googleapis.com \  
  cloudresourcemanager.googleapis.com \  
  serviceusage.googleapis.com \  
  discoveryengine.googleapis.com
```

API 簡介

- **Vertex AI API** (`aiplatform.googleapis.com`)：可使用 Gemini 生成內容，並使用 Vertex AI Embeddings 將文字向量化。
 - **Cloud SQL Admin API** (`sqladmin.googleapis.com`)：可讓您以程式輔助方式管理 Cloud SQL 執行個體。
 - **Discovery Engine API** (`discoveryengine.googleapis.com`)：支援 Vertex AI Reranker 功能。
 - **Service Usage API** (`serviceusage.googleapis.com`)：用於檢查及管理服務配額。
-

5. 建立虛擬環境並安裝依附元件

開始任何 Python 專案前，建議您先建立虛擬環境。這樣一來，專案的依附元件就會與其他專案或系統的全域 Python 套件隔離，避免發生衝突。

1. 建立名為 `rag-labs` 的資料夾，然後切換至該資料夾。在終端機中執行下列程式碼：

```
mkdir rag-labs && cd rag-labs
```

2. 建立並啟動虛擬環境：

```
uv venv --python 3.12
source .venv/bin/activate
```

3. 建立包含必要依附元件的 `requirements.txt` 檔案。在終端機中執行下列程式碼：

```
cloudshell edit requirements.txt
```

4. 將下列最佳化的依附元件貼到 `requirements.txt`。這些版本會固定，避免發生衝突並加快安裝速度。

```
# Core LangChain & AI
langchain-community==0.3.31
langchain-google-vertexai==2.1.2
langchain-google-community[vertexaisearch]==2.0.10

# Google Cloud
google-cloud-storage==2.19.0
google-cloud-aiplatform[langchain]==1.130.0

# Database
cloud-sql-python-connector[pg8000]==1.19.0
sqlalchemy==2.0.45
pgvector==0.4.2

# Utilities
tiktoken==0.12.0
python-dotenv==1.2.1
requests==2.32.5
```

5. 安裝依附元件：

```
uv pip install -r requirements.txt
```

6. 設定 PostgreSQL 適用的 Cloud SQL

在這項工作中，您將佈建 PostgreSQL 適用的 Cloud SQL 執行個體、建立資料庫，並準備進行向量搜尋。

定義 Cloud SQL 設定

1. 建立 `.env` 檔案來儲存設定。在終端機中執行下列程式碼：

```
cloudshell edit .env
```

2. 將下列設定貼到 `.env`。

```
# Project Config
PROJECT_ID="[YOUR_PROJECT_ID]"
REGION="us-central1"

# Database Config
SQL_INSTANCE_NAME="rag-pg-instance-1"
SQL_DATABASE_NAME="rag_harry_potter_db"
SQL_USER="rag_user"
SQL_PASSWORD="StrongPassword123!"

# RAG Config
PGVECTOR_COLLECTION_NAME="rag_harry_potter"
RANKING_LOCATION_ID="global"

# Connection Name (Auto-generated in scripts usually, but useful to have)
DB_INSTANCE_CONNECTION_NAME="${PROJECT_ID}:${REGION}:${SQL_INSTANCE_NAME}"
```

注意：如果是實際工作環境的應用程式，請使用 Google Cloud Secret Manager 儲存機密密碼。本實驗室使用 `.env` 檔案即可。

3. 將 `[YOUR_PROJECT_ID]` 替換為實際的 Google Cloud 專案 ID。(例如：`PROJECT_ID = "google-cloud-labs"`) 如果您不記得專案 ID，請在終端機中執行下列指令。畫面會列出所有專案及其 ID。

```
gcloud projects list
```

4. 將變數載入殼層工作階段：

```
source .env
```

建立執行個體和資料庫

1. 建立 PostgreSQL 適用的 Cloud SQL 執行個體。這個指令會建立適合本實驗室的小型執行個體。

```
gcloud sql instances create ${SQL_INSTANCE_NAME} \
  --database-version=POSTGRES_15 \
  --tier=db-g1-small \
  --region=${REGION} \
  --project=${PROJECT_ID}
```

注意：建立新的 Cloud SQL 執行個體約需 **5 到 7 分鐘**。敬請耐心等待。

- 執行個體準備就緒後，請建立資料庫：

```
gcloud sql databases create ${SQL_DATABASE_NAME} \
  --instance=${SQL_INSTANCE_NAME} \
  --project=${PROJECT_ID}
```

- 建立資料庫使用者：

```
gcloud sql users create ${SQL_USER} \
  --instance=${SQL_INSTANCE_NAME} \
  --password=${SQL_PASSWORD} \
  --project=${PROJECT_ID}
```

啟用 pgvector 擴充功能

`pgvector` 擴充功能可讓 PostgreSQL 儲存及搜尋向量嵌入。您必須在資料庫中明確啟用這項功能。

- 建立名為 `enable_pgvector.py` 的指令碼。在終端機中執行下列程式碼：

```
cloudshell edit enable_pgvector.py
```

- 將下列程式碼貼入 `enable_pgvector.py`。這項指令碼會連線至資料庫並執行 `CREATE EXTENSION IF NOT EXISTS vector;`。

```

import os
import sqlalchemy
from google.cloud.sql.connector import Connector, IPTypes
import logging
from dotenv import load_dotenv

load_dotenv()
logging.basicConfig(level=logging.INFO)

# Config
project_id = os.getenv("PROJECT_ID")
region = os.getenv("REGION")
instance_name = os.getenv("SQL_INSTANCE_NAME")
db_user = os.getenv("SQL_USER")
db_pass = os.getenv("SQL_PASSWORD")
db_name = os.getenv("SQL_DATABASE_NAME")
instance_connection_name = f"{project_id}:{region}:{instance_name}"

def getconn():
    with Connector() as connector:
        conn = connector.connect(
            instance_connection_name,
            "pg8000",
            user=db_user,
            password=db_pass,
            db=db_name,
            ip_type=IPTypes.PUBLIC,
        )
    return conn

def enable_pgvector():
    pool = sqlalchemy.create_engine(
        "postgresql+pg8000://",
        creator=getconn,
    )
    with pool.connect() as db_conn:
        # Check if extension exists
        result = db_conn.execute(sqlalchemy.text("SELECT extname FROM pg_extension WHERE
extname = 'vector';")).fetchone()
        if result:
            logging.info("pgvector extension is already enabled.")
        else:
            logging.info("Enabling pgvector extension...")
            db_conn.execute(sqlalchemy.text("CREATE EXTENSION IF NOT EXISTS vector;"))
            db_conn.commit()
            logging.info("pgvector extension enabled successfully.")

if __name__ == "__main__":
    enable_pgvector()

```

3. 執行指令碼：

```
python enable_pgvector.py
```

7. 第 1 部分：分塊策略

任何 RAG 管道的第一步，都是將文件轉換成 LLM 可解讀的格式：**區塊**。

LLM 的脈絡窗口有限 (一次可處理的文字量)。此外，如果為了回答特定問題而擷取 50 頁的文件，資訊就會變得不夠精確。我們會將文件分割成較小的「區塊」，以便找出相關資訊。

不過，*如何*分割文字非常重要：

- **字元分割器**：嚴格依字元數分割，這種做法速度很快，但風險很高，因為可能會將字詞或句子截斷，破壞語意。
- **遞迴分割器**：先嘗試依段落分割，然後依句子分割，最後依字詞分割。並盡量將語意單元放在一起。
- **權杖分割器**：根據 LLM 自身的詞彙 (權杖) 分割。這可確保區塊完全符合脈絡視窗，但生成成本可能較高。

在本節中，您將使用這三種策略擷取相同資料，以便進行比較。

建立擷取指令碼

您將使用指令碼下載哈利波特資料集，並透過「Character」(字元)、「Recursive」(遞迴) 和「Token」(權杖) 策略分割資料集，然後將嵌入內容上傳至 Cloud SQL 中的三個不同資料表。

1. 建立 `ingest_data.py` 檔案：

```
cloudshell edit ingest_data.py
```

2. 將下列**修正**程式碼貼到 `ingest_data.py`。這個版本可正確剖析資料集的 JSON 結構。

```

import os
import json
import logging
import requests
from typing import List, Dict, Any
from dotenv import load_dotenv

from google.cloud.sql.connector import Connector, IPTypes
from langchain_google_vertexai import VertexAIEmbeddings
from langchain_community.vectorstores import PGVector
from langchain.text_splitter import CharacterTextSplitter,
RecursiveCharacterTextSplitter, TokenTextSplitter
from langchain.docstore.document import Document

load_dotenv()
logging.basicConfig(level=logging.INFO)

# Configuration
PROJECT_ID = os.getenv("PROJECT_ID")
REGION = os.getenv("REGION")
DB_USER = os.getenv("SQL_USER")
DB_PASS = os.getenv("SQL_PASSWORD")
DB_NAME = os.getenv("SQL_DATABASE_NAME")
INSTANCE_CONNECTION_NAME = f"{PROJECT_ID}:{REGION}:{os.getenv('SQL_INSTANCE_NAME')}"
BASE_COLLECTION_NAME = os.getenv("PGVECTOR_COLLECTION_NAME")
BOOKS_JSON_URL = "https://storage.googleapis.com/github-repo/generative-
ai/gemini/reasoning-engine/sample_data/harry_potter_books.json"

CHUNK_SIZE = 500
CHUNK_OVERLAP = 50
MAX_DOCS_TO_PROCESS = 10

# Database Connector
def getconn():
    with Connector() as connector:
        return connector.connect(
            INSTANCE_CONNECTION_NAME,
            "pg8000",
            user=DB_USER,
            password=DB_PASS,
            db=DB_NAME,
            ip_type=IPTypes.PUBLIC,
        )

def download_data():
    logging.info(f"Downloading data from {BOOKS_JSON_URL}...")
    response = requests.get(BOOKS_JSON_URL)
    return response.json()

def prepare_chunks(json_data, strategy):
    documents = []

```

```

# Iterate through the downloaded data
for entry in json_data[:MAX_DOCS_TO_PROCESS]:

    # --- JSON PARSING LOGIC ---
    # The data structure nests content inside 'kwargs' -> 'page_content'
    if "kwargs" in entry and "page_content" in entry["kwargs"]:
        content = entry["kwargs"]["page_content"]

        # Extract metadata if available, ensuring it's a dict
        metadata = entry["kwargs"].get("metadata", {})
        if not isinstance(metadata, dict):
            metadata = {"source": "unknown"}

        # Add the strategy to metadata for tracking
        metadata["strategy"] = strategy
    else:
        continue

    if not content:
        continue

    # Choose the splitter based on the strategy
    if strategy == "character":
        splitter = CharacterTextSplitter(chunk_size=CHUNK_SIZE,
chunk_overlap=CHUNK_OVERLAP, separator="\n")
    elif strategy == "token":
        splitter = TokenTextSplitter(chunk_size=CHUNK_SIZE,
chunk_overlap=CHUNK_OVERLAP)
    else: # default to recursive
        splitter = RecursiveCharacterTextSplitter(chunk_size=CHUNK_SIZE,
chunk_overlap=CHUNK_OVERLAP)

    # Split the content into chunks
    chunks = splitter.split_text(content)

    # Create Document objects for each chunk
    for chunk in chunks:
        documents.append(Document(page_content=chunk, metadata=metadata))

return documents

def main():
    logging.info("Initializing Embeddings...")
    embeddings = VertexAIEmbeddings(model_name="gemini-embedding-001",
project=PROJECT_ID, location=REGION)

    data = download_data()
    strategies = ["character", "recursive", "token"]

    # Connection string for PGVector (uses the getconn helper)
    pg_conn_str = f"postgresql+pg8000://{DB_USER}:{DB_PASS}@placeholder/{DB_NAME}"

```

```

for strategy in strategies:
    collection_name = f"{BASE_COLLECTION_NAME}_{strategy}"
    logging.info(f"--- Processing strategy: {strategy.upper()} ---")
    logging.info(f"Target Collection: {collection_name}")

    # Prepare documents with the specific strategy
    docs = prepare_chunks(data, strategy)

    if not docs:
        logging.warning(f"No documents generated for strategy {strategy}. Check data
source.")
        continue

    logging.info(f"Generated {len(docs)} chunks. Uploading to Cloud SQL...")

    # Initialize the Vector Store
    store = PGVector(
        collection_name=collection_name,
        embedding_function=embeddings,
        connection_string=pg_conn_str,
        engine_args={"creator": getconn},
        pre_delete_collection=True # Clears old data for this collection before
adding new
    )

    # Batch add documents
    store.add_documents(docs)
    logging.info(f"Successfully finished {strategy}.\n")

if __name__ == "__main__":
    main()

```

3. 執行擷取指令碼。這樣資料庫就會填入三個不同的資料表 (集合)。

```
python ingest_data.py
```

比較分塊結果

資料載入完畢後，請針對這三個集合執行查詢，瞭解分塊策略對結果的影響。

1. 建立 `query_chunking.py`：

```
cloudshell edit query_chunking.py
```

2. 將下列程式碼貼入 `query_chunking.py`：

```

import os
import logging
from dotenv import load_dotenv
from google.cloud.sql.connector import Connector, IPTypes
from langchain_google_vertexai import VertexAIEmbeddings
from langchain_community.vectorstores import PGVector

load_dotenv()
logging.basicConfig(level=logging.ERROR) # Only show errors to keep output clean

# Config
PROJECT_ID = os.getenv("PROJECT_ID")
REGION = os.getenv("REGION")
DB_USER = os.getenv("SQL_USER")
DB_PASS = os.getenv("SQL_PASSWORD")
DB_NAME = os.getenv("SQL_DATABASE_NAME")
INSTANCE_CONNECTION_NAME = f"{PROJECT_ID}:{REGION}:{os.getenv('SQL_INSTANCE_NAME')}"
BASE_COLLECTION_NAME = os.getenv("PGVECTOR_COLLECTION_NAME")

def getconn():
    with Connector() as connector:
        return connector.connect(
            INSTANCE_CONNECTION_NAME,
            "pg8000",
            user=DB_USER,
            password=DB_PASS,
            db=DB_NAME,
            ip_type=IPTypes.PUBLIC,
        )

def main():
    embeddings = VertexAIEmbeddings(model_name="gemini-embedding-001",
project=PROJECT_ID, location=REGION)
    pg_conn_str = f"postgresql+pg8000://{DB_USER}:{DB_PASS}@placeholder/{DB_NAME}"

    query = "Tell me about the Dursleys and their relationship with Harry Potter"
    print(f"\nQUERY: {query}\n" + "="*50)

    strategies = ["character", "recursive", "token"]

    for strategy in strategies:
        collection = f"{BASE_COLLECTION_NAME}_{strategy}"
        print(f"\nSTRATEGY: {strategy.upper()}")

        store = PGVector(
            collection_name=collection,
            embedding_function=embeddings,
            connection_string=pg_conn_str,
            engine_args={"creator": getconn}
        )

        results = store.similarity_search_with_score(query, k=2)

```

```
for i, (doc, score) in enumerate(results):
    print(f"  Result {i+1} (Score: {score:.4f}):
{doc.page_content[:150].replace(chr(10), ' ')}...")

if __name__ == "__main__":
    main()
```

3. 執行查詢指令碼：

```
python query_chunking.py
```

查看輸出內容。

請注意，**Character** 分割可能會在句子中途截斷，而 **Recursive** 則會盡量尊重段落界線。**權杖**分割可確保區塊完全符合 LLM 脈絡窗口，但可能會忽略語意結構。

8. 第 2 部分：重新排序

向量搜尋 (檢索) 速度極快，因為這項功能採用壓縮的數學表示法 (嵌入)。這項模型會廣泛搜尋，確保召回率 (找出所有可能相關的項目)，但通常精確度較低 (這些項目的排名可能不完美)。

相關文件通常會「迷失在中間」的結果清單中。如果 LLM 只關注前 5 個結果，可能會錯過第 7 個位置的重要答案。

重新排序會新增第二階段，解決這個問題。

1. **檢索器**：使用快速向量搜尋擷取較大的集合 (例如前 25 個)。
2. **重新排序器**：使用專用模型 (例如 Cross-Encoder) 檢查查詢和文件配對的全文。雖然速度較慢，但準確度高出許多。系統會重新評估前 25 名，並傳回絕對最佳的 3 個結果。

在這項工作中，您將搜尋第 1 部分建立的 `recursive` 集合，但這次會套用 **Vertex AI Reranker** 來修正結果。

1. 建立 `query_reranking.py`：

```
cloudshell edit query_reranking.py
```

2. 貼上下列程式碼。請注意，這個函式會明確指定 `_recursive` 集合，並使用 `ContextualCompressionRetriever`。

```

import os
import logging
from dotenv import load_dotenv
from google.cloud.sql.connector import Connector, IPTypes
from langchain_google_vertexai import VertexAIEmbeddings
from langchain_community.vectorstores import PGVector

# Reranking Imports
from langchain.retrievers import ContextualCompressionRetriever
from langchain_google_community.vertex_rank import VertexAIRank

load_dotenv()
logging.basicConfig(level=logging.ERROR)

PROJECT_ID = os.getenv("PROJECT_ID")
REGION = os.getenv("REGION")
DB_USER = os.getenv("SQL_USER")
DB_PASS = os.getenv("SQL_PASSWORD")
DB_NAME = os.getenv("SQL_DATABASE_NAME")
INSTANCE_CONNECTION_NAME = f"{PROJECT_ID}:{REGION}:{os.getenv('SQL_INSTANCE_NAME')}"

# IMPORTANT: Target the recursive collection created in ingest_data.py
COLLECTION_NAME = f"{os.getenv('PGVECTOR_COLLECTION_NAME')}_recursive"
RANKING_LOCATION = os.getenv("RANKING_LOCATION_ID")

def getconn():
    with Connector() as connector:
        return connector.connect(
            INSTANCE_CONNECTION_NAME,
            "pg8000",
            user=DB_USER,
            password=DB_PASS,
            db=DB_NAME,
            ip_type=IPTypes.PUBLIC,
        )

def main():
    embeddings = VertexAIEmbeddings(model_name="gemini-embedding-001",
    project=PROJECT_ID, location=REGION)
    pg_conn_str = f"postgresql+pg8000://{DB_USER}:{DB_PASS}@placeholder/{DB_NAME}"

    print(f"Connecting to collection: {COLLECTION_NAME}")
    store = PGVector(
        collection_name=COLLECTION_NAME,
        embedding_function=embeddings,
        connection_string=pg_conn_str,
        engine_args={"creator": getconn}
    )

    query = "What are the Horcruxes?"
    print(f"QUERY: {query}\n")

```

```

# 1. Base Retriever (Vector Search) - Fetch top 10
base_retriever = store.as_retriever(search_kwargs={"k": 10})

# 2. Reranker - Select top 3 from the 10
reranker = VertexAIRank(
    project_id=PROJECT_ID,
    location_id=RANKING_LOCATION,
    ranking_config="default_ranking_config",
    title_field="source",
    top_n=3
)

compression_retriever = ContextualCompressionRetriever(
    base_compressor=reranker,
    base_retriever=base_retriever
)

# Execute
try:
    reranked_docs = compression_retriever.invoke(query)

    if not reranked_docs:
        print("No documents returned. Check if the collection exists and is
populated.")

    print(f"--- Top 3 Reranked Results ---")
    for i, doc in enumerate(reranked_docs):
        print(f"Result {i+1} (Score: {doc.metadata.get('relevance_score', 'N/A')}):")
        print(f"    {doc.page_content[:200]}...\n")
except Exception as e:
    print(f"Error during reranking: {e}")

if __name__ == "__main__":
    main()

```

3. 執行重新排序查詢：

```
python query_reranking.py
```

觀察

您可能會發現，與原始向量搜尋相比，關聯性分數較高或排序不同。確保 LLM 獲得最精確的脈絡資訊。

9. 第 3 部分：查詢轉換

通常，RAG 的最大瓶頸在於使用者。使用者查詢通常模稜兩可、不完整或措辭不當。如果查詢嵌入與文件嵌入在數學上不一致，擷取就會失敗。

查詢轉換功能會使用 LLM 在查詢傳送至資料庫前重寫或擴充查詢。您將實作兩種技術：

- **HyDE (假設性文件嵌入)**：問題和答案之間的向量相似度，通常會低於答案和假設性答案之間的相似度。HyDE 會要求 LLM 虛構出完美答案，然後嵌入該答案，並搜尋與虛構答案相似的文件。
- **退回提示**：如果使用者提出具體詳細的問題，系統可能會忽略更廣泛的背景資訊。「退一步」提示會要求 LLM 生成更高層級的抽象問題 (例如「這個家庭的歷史是什麼？」)，以便擷取基礎資訊和具體細節。

1. 建立 `query_transformation.py`：

```
cloudshell edit query_transformation.py
```

2. 貼上下列程式碼：

```

import os
import logging
from dotenv import load_dotenv
from google.cloud.sql.connector import Connector, IPTypes
from langchain_google_vertexai import VertexAIEmbeddings, VertexAI
from langchain_community.vectorstores import PGVector
from langchain_core.prompts import PromptTemplate

load_dotenv()
logging.basicConfig(level=logging.ERROR)

PROJECT_ID = os.getenv("PROJECT_ID")
REGION = os.getenv("REGION")
DB_USER = os.getenv("SQL_USER")
DB_PASS = os.getenv("SQL_PASSWORD")
DB_NAME = os.getenv("SQL_DATABASE_NAME")
INSTANCE_CONNECTION_NAME = f"{PROJECT_ID}:{REGION}:{os.getenv('SQL_INSTANCE_NAME')}"
COLLECTION_NAME = f"{os.getenv('PGVECTOR_COLLECTION_NAME')}_recursive"

def getconn():
    with Connector() as connector:
        return connector.connect(
            INSTANCE_CONNECTION_NAME,
            "pg8000",
            user=DB_USER,
            password=DB_PASS,
            db=DB_NAME,
            ip_type=IPTypes.PUBLIC,
        )

def generate_hyde_doc(query, llm):
    prompt = PromptTemplate(
        input_variables=["question"],
        template="Write a concise, hypothetical answer to the question. Question: {question} Answer:"
    )
    chain = prompt | llm
    return chain.invoke({"question": query})

def generate_step_back(query, llm):
    prompt = PromptTemplate(
        input_variables=["question"],
        template="Write a more general, abstract question that concepts in this question. Original: {question} Step-back:"
    )
    chain = prompt | llm
    return chain.invoke({"question": query})

def main():
    embeddings = VertexAIEmbeddings(model_name="gemini-embedding-001",
    project=PROJECT_ID, location=REGION)
    llm = VertexAI(model_name="gemini-2.5-flash", project=PROJECT_ID, location=REGION,

```

```

temperature=0.5)

pg_conn_str = f"postgresql+pg8000://{DB_USER}:{DB_PASS}@placeholder/{DB_NAME}"
store = PGVector(
    collection_name=COLLECTION_NAME,
    embedding_function=embeddings,
    connection_string=pg_conn_str,
    engine_args={"creator": getconn}
)
retriever = store.as_retriever(search_kwargs={"k": 2})

original_query = "Tell me about the Dursleys."
print(f"ORIGINAL QUERY: {original_query}\n" + "-"*30)

# 1. HyDE
hyde_doc = generate_hyde_doc(original_query, llm)
print(f"HyDE Generated Doc: {hyde_doc.strip()[:100]}...")
hyde_results = retriever.invoke(hyde_doc)
print(f"HyDE Retrieval: {hyde_results[0].page_content[:100]}...\n")

# 2. Step-back
step_back_q = generate_step_back(original_query, llm)
print(f"Step-back Query: {step_back_q.strip()}")
step_results = retriever.invoke(step_back_q)
print(f"Step-back Retrieval: {step_results[0].page_content[:100]}...")

if __name__ == "__main__":
    main()

```

3. 執行轉換指令碼：

```
python query_transformation.py
```

查看輸出內容。

請注意，「Step-back」查詢可能會擷取有關德思禮家族史的廣泛脈絡，而「HyDE」則著重於假設性答案中生成的特定詳細資料。

10. 第 4 部分：端對端生成

我們已將資料切成小塊、精簡搜尋範圍，並修正使用者的查詢。現在，我們終於要加入 RAG 中的「G」：生成。

到目前為止，我們只尋找資訊，如要打造真正的 AI 助理，我們需要將這些經過重新排序的高品質文件提供給大型語言模型 (Gemini)，以便合成自然語言答案。

在實際工作環境管道中，這涉及特定流程：

1. **擷取**：使用快速向量搜尋功能，取得大量候選項目 (例如前 10 名)。
2. **重新排序**：使用 Vertex AI Reranker 篩選出最佳結果 (例如前 3 名)。
3. **建構背景資訊**：將前 3 份文件的內容合併為單一字串。
4. **根據事實的提示**：將該背景資訊字串插入嚴格的提示範本，強制 LLM 「只」使用該資訊。

建立生成指令碼

我們會使用 `gemini-2.5-flash` 執行生成步驟。這個模型具有長脈絡窗口和低延遲的特性，因此非常適合 RAG，可快速處理多個擷取的檔案。

1. 建立 `end_to_end_rag.py`：

```
cloudshell edit end_to_end_rag.py
```

2. 貼上下列程式碼。請注意 `template` 變數，我們會嚴格指示模型將其繫結至提供的內容，避免「幻覺」(編造內容)。

```

import os
import logging
from dotenv import load_dotenv
from google.cloud.sql.connector import Connector, IPTypes
from langchain_google_vertexai import VertexAIEmbeddings, VertexAI
from langchain_community.vectorstores import PGVector
from langchain.retrievers import ContextualCompressionRetriever
from langchain_google_community.vertex_rank import VertexAIRank
from langchain.prompts import PromptTemplate
from langchain.chains import LLMChain

load_dotenv()
logging.basicConfig(level=logging.ERROR)

PROJECT_ID = os.getenv("PROJECT_ID")
REGION = os.getenv("REGION")
# We use the recursive collection as it generally provides the best context boundaries
COLLECTION_NAME = f"{os.getenv('PGVECTOR_COLLECTION_NAME')}_recursive"

def getconn():
    instance_conn = f"{PROJECT_ID}:{REGION}:{os.getenv('SQL_INSTANCE_NAME')}"
    with Connector() as connector:
        return connector.connect(
            instance_conn, "pg8000",
            user=os.getenv("SQL_USER"), password=os.getenv("SQL_PASSWORD"),
            db=os.getenv("SQL_DATABASE_NAME"), ip_type=IPTypes.PUBLIC
        )

def main():
    print("--- Initializing Production RAG Pipeline ---")

    # 1. Setup Embeddings (Gemini Embedding 001)
    # We use this to vectorize the user's query to match our database.
    embeddings = VertexAIEmbeddings(model_name="gemini-embedding-001", project=PROJECT_ID,
location=REGION)

    # 2. Connect to Vector Store
    pg_conn_str = f"postgresql+pg8000://{os.getenv('SQL_USER')}:
{os.getenv('SQL_PASSWORD')}@placeholder/{os.getenv('SQL_DATABASE_NAME')}"
    store = PGVector(
        collection_name=COLLECTION_NAME,
        embedding_function=embeddings,
        connection_string=pg_conn_str,
        engine_args={"creator": getconn}
    )

    # 3. Setup The 'Filter Funnel' (Retriever + Reranker)
    # Step A: Fast retrieval of top 10 similar documents
    base_retriever = store.as_retriever(search_kwargs={"k": 10})

    # Step B: Precise reranking to find the top 3 most relevant
    reranker = VertexAIRank(

```

```

        project_id=PROJECT_ID,
        location_id="global",
        ranking_config="default_ranking_config",
        title_field="source",
        top_n=3
    )

# Combine A and B into a single retrieval object
compression_retriever = ContextualCompressionRetriever(
    base_compressor=reranker,
    base_retriever=base_retriever
)

# 4. Setup LLM (Gemini 2.5 Flash)
# We use a low temperature (0.1) to reduce creativity and increase factual adherence.
llm = VertexAI(model_name="gemini-2.5-flash", project=PROJECT_ID, location=REGION,
temperature=0.1)

# --- Execution Loop ---
user_query = "Who is Harry Potter?"
print(f"\nUser Query: {user_query}")
print("Retrieving and Reranking documents...")

# Retrieve the most relevant documents
top_docs = compression_retriever.invoke(user_query)

if not top_docs:
    print("No relevant documents found.")
    return

# Build the Context String
# We stitch the documents together, labeling them as Source 1, Source 2, etc.
context_str = "\n\n".join([f"Source {i+1}: {d.page_content}" for i, d in
enumerate(top_docs)])

print(f"Found {len(top_docs)} relevant context chunks.")

# 5. The Grounded Prompt
template = """You are a helpful assistant. Answer the question strictly based on the
provided context.
If the answer is not in the context, say "I don't know."

Context:
{context}

Question:
{question}

Answer:
"""

prompt = PromptTemplate(template=template, input_variables=["context", "question"])

```

```
# Create the chain: Prompt -> LLM
chain = prompt | llm

print("Generating Answer via Gemini 2.5 Flash...")
final_answer = chain.invoke({"context": context_str, "question": user_query})

print(f"\nFINAL ANSWER:\n{final_answer}")

if __name__ == "__main__":
    main()
```

3. 執行最終應用程式：

```
python end_to_end_rag.py
```

瞭解輸出內容

執行這項指令碼時，請觀察擷取的原始區塊 (您在先前的步驟中看到) 與最終答案之間的差異。LLM 會充當合成器，讀取 Reranker 提供的文字「區塊」，並將這些區塊整合成連貫且易於閱讀的句子。

透過串連這些元件，您可以從隨機「猜測」轉移至確定性工作流程。**檢索器**會撒網，**重新排序器**會選取最佳結果，**生成器**則會烹調餐點。

11. 結論

恭喜！您已成功建構進階 RAG 管道，遠遠超出基本向量搜尋的範圍。

重點回顧

- 您已設定 **Cloud SQL** 和 **pgvector**，以便擴充向量儲存空間。
- 您比較了分塊策略，瞭解資料準備對擷取的影響。
- 您已透過 Vertex AI 實作**重新排序**功能，提升結果的準確度。
- 您使用**查詢轉換** (HyDE、Step-back) 將使用者意圖與資料對齊。

瞭解詳情

- [RAG 參考架構](#)：瀏覽 RAG 相關參考架構指南清單。

從原型設計到投入正式環境

這個實驗室屬於「**Google Cloud 學習路徑：打造可用於正式環境的 AI**」。

- [探索完整課程](#)，從設計原型開始，一步步助您把專案投入正式環境。
 - 使用 **#ProductionReadyAI** 主題標記分享你的進度。
-